

Tổng quan chương.

Chương 2 đã trình bày nền tảng của MLS, TreeKEM, mạng ngang hàng và nguyên nhân dẫn tới phân nhánh trạng thái mật mã khi nhiều Commit cùng xuất hiện trong một Epoch. Trên cơ sở đó, Chương 3 tập trung vào lớp điều phối phi tập trung mà đề án đề xuất để MLS có thể vận hành trong môi trường P2P mà không cần một Delivery Service trung tâm sắp thứ tự Commit.

Chương này trình bày phương pháp đề xuất ở mức giao thức, bao gồm: bài toán và yêu cầu thiết kế, cơ chế Single-Writer theo epoch, cơ chế kiểm tra nhất quán epoch, cơ chế phát hiện và Fork Healing, và cách dùng đồng hồ logic lai để sắp xếp thông điệp ứng dụng. Cuối chương có một mục ngắn nói về cách ứng dụng thử nghiệm hiện thực các nguyên tắc này; phần kiểm chứng và ảnh minh họa nằm ở Chương 5 và phụ lục.

0.1 Tổng quan bài toán và hướng tiếp cận

0.1.1 Bài toán cần giải quyết

Trong mô hình MLS thông thường, Delivery Service không chỉ chuyển tiếp thông điệp mà còn giúp các thành viên nhìn thấy cùng một thứ tự Commit **rfc9420**, **rfc9750**. Khi chuyển sang môi trường P2P và bỏ điểm tuần tự hóa trung tâm này, hệ thống phải đối mặt với một rủi ro trực tiếp: hai hoặc nhiều thành viên có thể cùng phát hành Commit cho cùng một Epoch. Khi đó, nhóm không còn tiến hóa theo một chuỗi trạng thái duy nhất, mà có thể tách thành nhiều nhánh trạng thái mật mã khác nhau.

Giả sử nhóm đang ở Epoch E . Nếu thành viên A tạo Commit C_A và thành viên B đồng thời tạo Commit C_B , một phần nút có thể áp dụng C_A trước, trong khi phần còn lại áp dụng C_B trước. Kết quả là nhóm có thể sinh ra hai trạng thái $S_A(E+1)$ và $S_B(E+1)$ khác nhau dù cùng mang nhãn Epoch $E+1$. Khi đó, các nút ở hai nhánh không còn cùng tree hash, cùng lịch sử Commit hay cùng khóa ứng dụng, nên cũng không còn xử lý thông điệp của nhau một cách hợp lệ **alwen2023fork**, **kohbrok2025dmls**.

Vì vậy, phương pháp đề xuất trong đề án phải đồng thời đáp ứng bốn yêu cầu: (i) giảm mạnh khả năng xuất hiện concurrent commits khi mạng còn liên thông; (ii) buộc mỗi nút chỉ xử lý thông điệp trên trạng thái MLS tương thích với epoch cục bộ của nó; (iii) phát hiện được sự phân kỳ trạng thái khi phân vùng mạng xảy ra và hỗ trợ hội tụ trở lại một nhánh hợp lệ; (iv) giữ được một thứ tự hiển thị ổn định cho các thông điệp ứng dụng trong cùng một Epoch mà không nhầm lẫn vai trò đó với thứ tự Commit.

0.1.2 Kiến trúc của giải pháp

Về kiến trúc, phương pháp chia thành hai tầng. Tầng điều phối quyết định khi nào một nút được quyền tạo Commit, cách phân loại thông điệp theo epoch, cách đối chiếu trạng thái nhánh và cách khôi phục sau phân vùng mạng. Tầng MLS lo việc xử lý Proposal, Commit, Welcome, External Proposal, External Commit, mã hóa và giải mã thông điệp theo đúng quy tắc của MLS **rfc9420**.

Hình 0.1 cho thấy cách phân chia trách nhiệm này: lớp điều phối không lập trình lại MLS, còn lõi MLS không tự quyết định thứ tự điều phối trong môi trường phân tán. Nhờ cách tách này, đề án giữ nguyên các bảo đảm mật mã của MLS và chỉ bổ sung lớp điều phối cho bối cảnh P2P.

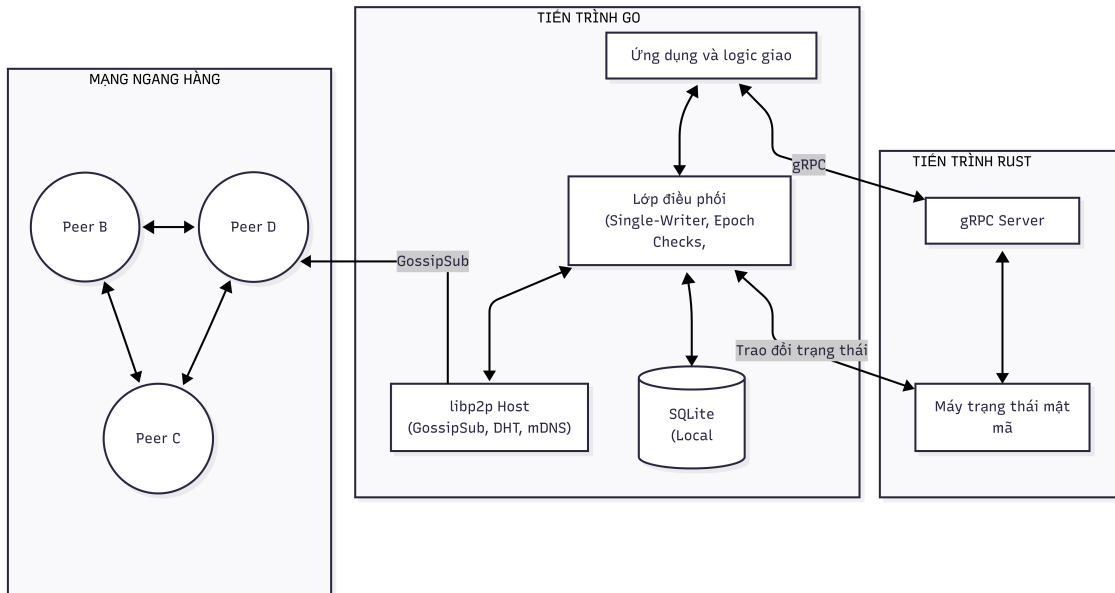


Figure 0.1: Kiến trúc logic của giải pháp và ranh giới trách nhiệm giữa lớp điều phối với lớp MLS

0.1.3 Các cơ chế chính của phương pháp

Phương pháp đề xuất gồm bốn cơ chế liên kết với nhau: (1) Single-Writer theo epoch để giới hạn quyền phát hành Commit; (2) kiểm tra nhất quán epoch để ngăn nút xử lý thông điệp trên trạng thái MLS không tương thích; (3) phát hiện và Fork Healing để đưa các nút hội tụ lại sau phân vùng mạng; (4) đồng hồ logic lai HLC để sắp xếp thông điệp ứng dụng sau khi đã qua lớp kiểm tra của MLS.

Bốn cơ chế này không thay thế MLS mà chỉ bao quanh nó, bù vào vai trò mà Delivery Service thường đảm nhiệm trong mô hình tập trung. Đóng góp của đề án là cách tổ chức lớp điều phối chứ không phải sửa đổi thuật toán mật mã cốt lõi.

0.1.4 Phạm vi bảo mật và Mô hình mối đe dọa (Threat Model)

Vì đồ án sử dụng mạng P2P thay cho máy chủ trung tâm, rủi ro bảo mật thực chất lại mở rộng hơn do mọi nút trong mạng đều có thể tham gia định tuyến tin nhắn và tiếp cận siêu dữ liệu (metadata). Để làm rõ ranh giới giữa an toàn mật mã và đồng thuận phân tán, đồ án xác định mô hình mối đe dọa gồm ba đối tượng chính:

- **Kẻ nghe lén (Passive Eavesdropper):** Bất kỳ nút P2P trung gian nào cố tình theo dõi luồng dữ liệu truyền qua. Rủi ro này được ủy thác hoàn toàn cho lớp mật mã: hệ thống chống lại bằng cơ chế mã hóa có xác thực (AEAD) của MLS [rfc5116](#), đảm bảo trung gian tuyệt đối không thể truy cập bản rõ.
- **Thành viên bị chiếm quyền (Compromised Member):** Một thành viên hợp lệ trong nhóm bị kẻ tấn công đánh cắp thiết bị hoặc lộ vật liệu khóa. Hệ thống kế thừa và chống lại rủi ro này bằng các thuộc tính Forward Secrecy (FS) và Post-Compromise Security (PCS) mặc định của cơ chế TreeKEM trong MLS [rfc9420](#).
- **Kẻ phá hoại mạng (Active Attacker / Malicious Node):** Nút mạng cố tình giữ lại tin nhắn, phát tán thông điệp sai thứ tự, hoặc cố ý tạo phân nhánh nhằm chia rẽ sự đồng thuận của nhóm. Đây chính là trọng tâm mà lớp điều phối phi tập trung của đồ án trực tiếp giải quyết thông qua cơ chế bầu Token Holder (Single-Writer) và quá trình hàn gắn phân nhánh (Fork Healing).

Nói cách khác, đồ án giả định các cấu trúc mật mã của MLS là an toàn. Bài toán bảo mật được giải quyết bằng việc tích hợp chuẩn này; phần việc của đồ án là giữ cho hệ thống không bị sụp đổ hay phân nhánh khi mạng ngang hàng hoạt động bất đồng bộ.

0.2 Giao thức Single-Writer theo epoch

0.2.1 Mục tiêu thiết kế

Tiến hóa trạng thái nhóm về bản chất là một bài toán Máy trạng thái nhân bản (SMR) [schneider1990smr](#), nên thiết kế giao thức bị chi phối bởi các đánh đổi của định lý CAP [gilbert2002brewers](#). Nếu cố đạt tính Nhất quán mạnh như các thuật toán đồng thuận truyền thống, hệ thống nhắn tin P2P sẽ mất tính Khả dụng mỗi khi mạng bị chia cắt.

Đồ án chọn hướng ưu tiên tính Khả dụng và chịu đựng phân vùng mạng (AP), kết hợp với mô hình Nhất quán cuối cùng [vogels2008eventual](#). Nguyên nhân trực tiếp của phân nhánh là nhiều Commit hợp lệ cùng xuất hiện trong một epoch, nên đồ án kiểm soát bằng cách chỉ cho một thành viên duy nhất phát hành Commit.

Nếu mạng bị chia cắt và thông tin về Token bị phân mảnh, sinh ra nhiều nhánh Commit độc lập, hệ thống cho phép các nhóm con tiếp tục hoạt động, sau đó dùng cơ chế hàn gắn để đưa nhóm về một trạng thái chung khi mạng ổn định lại.

Ý tưởng trung tâm của cơ chế này là tách vai trò Proposal và Commit. Thành viên hợp lệ nào cũng có thể phát biểu mong muốn thay đổi nhóm dưới dạng Proposal, nhưng chỉ một thành viên duy nhất trong tập ứng viên hợp lệ mới được quyền gom các Proposal đó để tạo Commit. Thành viên này được gọi là Token Holder của Epoch đang xét.

0.2.2 Tập ứng viên hợp lệ

Token Holder không được chọn từ toàn bộ các nút mà hệ thống nhìn thấy trên mạng, mà chỉ được chọn từ tập ứng viên hợp lệ của nhóm. Tập này được xác định bởi ba loại điều kiện: tư cách thành viên trong trạng thái MLS, tình trạng còn hoạt động trên mạng và chính sách quyền của ứng dụng. Ở mức khái niệm, tập ứng viên tại Epoch E được mô tả như sau:

$$Eligible(E) = \mathcal{M}(E) \cap \mathcal{A}(E) \cap \mathcal{C}(E) \setminus \mathcal{S}(E) \quad (1)$$

Trong đó, $\mathcal{M}(E)$ là tập thành viên mà trạng thái MLS còn xem là hợp lệ ở Epoch E ; $\mathcal{A}(E)$ là tập thành viên đang được quan sát là còn hoạt động; $\mathcal{C}(E)$ là tập thành viên mà chính sách ứng dụng cho phép tham gia tạo Commit; còn $\mathcal{S}(E)$ là tập thành viên tạm thời bị loại khỏi vai trò điều phối vì lỗi, ngoại tuyến quá lâu hoặc bị chính sách ứng dụng đình chỉ.

Cách mô hình hóa này giúp phân biệt rõ hai chuyện thường bị trộn lẫn. Một thành viên có thể vẫn còn trong nhóm ở tầng MLS nhưng đang ngoại tuyến, nên không thích hợp giữ vai trò Token Holder. Ngược lại, một nút đang trực tuyến nhưng không thuộc trạng thái nhóm cũng không có quyền tham gia quyết định Commit. Do đó, chỉ phần giao của các điều kiện trên mới có ý nghĩa đối với bài toán điều phối.

0.2.3 Hàm bầu chọn Token Holder

Sau khi xác định được tập ứng viên hợp lệ, mỗi nút tự tính Token Holder bằng cùng một quy tắc tất định cục bộ. Thành viên có giá trị băm nhỏ nhất khi ghép định danh nhóm, số epoch và định danh nút sẽ được chọn:

$$TH(E) = \operatorname{argmin}_{n \in Eligible(E)} H(\text{group_id} \parallel E \parallel n) \quad (2)$$

Ở đây, H là hàm băm mật mã như SHA-256. Việc đưa group_id vào đầu vào

giúp một nút không mặc nhiên giữ ưu thế ở mọi nhóm; việc đưa E vào đầu vào giúp quyền giữ Token thay đổi theo từng epoch; còn định danh nút bảo đảm mọi nút đang có cùng ActiveView sẽ đi tới cùng kết quả. Thuật toán bầu chọn được mô tả trong Thuật toán 1.

Algorithm 1: Thuật toán bầu chọn Token Holder (*ComputeTokenHolder*)

Đầu vào: *groupID*: định danh nhóm; *epoch*: Epoch hiện tại; *members*: tập thành viên MLS; *activeView*: tập thành viên đang hoạt động; *authorized*: tập thành viên có quyền tạo Commit; *suspended*: tập thành viên bị đình chỉ tạm thời.

Đầu ra : *holderID*: định danh Token Holder được chọn.

$eligible \leftarrow members \cap activeView \cap authorized \setminus suspended$;

if *eligible* là tập rỗng **then**

return Lỗi *NoEligibleCommitter*;

end

$bestID \leftarrow \text{null}$;

$bestHash \leftarrow \text{MAX_HASH}$;

for mỗi *peerID* trong *eligible* **do**

$data \leftarrow \text{Encode}(groupID, epoch, peerID)$;

$candidateHash \leftarrow \text{SHA256}(data)$;

if $candidateHash < bestHash$ **then**

$bestHash \leftarrow candidateHash$;

$bestID \leftarrow peerID$;

end

end

return *bestID*;

Ưu điểm chính của cách chọn này là không cần thêm một vòng bỏ phiếu mạng cho mỗi Commit. Nếu các nút có cùng trạng thái nhóm và cùng ActiveView, chúng sẽ tự tính ra cùng một Token Holder. Chi phí của bước bầu chọn vì thế chủ yếu là chi phí cục bộ, tăng tuyến tính theo số lượng ứng viên hợp lệ.

0.2.4 Tạo Proposal và Commit

Trong cơ chế đề xuất, Proposal là yêu cầu thay đổi nhóm, còn Commit là thao tác thực sự làm nhóm chuyển sang Epoch mới **rfc9420**. Tất cả thành viên hợp lệ đều có thể tạo Proposal. Tuy nhiên, chỉ Token Holder của Epoch hiện tại mới có quyền gom các Proposal hợp lệ để phát hành Commit.

Quy tắc này cho phép hệ thống chấp nhận nhiều ý định thay đổi cùng lúc nhưng vẫn duy trì một nút duy nhất cho mỗi epoch được quyền Commit. Khi mạng còn liên thông, các nút sẽ có cùng Token Holder và vì thế giảm mạnh khả năng sinh concurrent commits. Quy trình tạo Commit của Token Holder được mô tả trong Thuật toán 2.

Algorithm 2: Quy trình tạo Commit của Token Holder
(*CommitByTokenHolder*)

Đầu vào: *localState*: trạng thái MLS cục bộ tại Epoch E ; *proposalQueue*: hàng đợi Proposal hợp lệ; *holderID*: Token Holder tại Epoch E ; *selfID*: định danh nút cục bộ.

Đầu ra : Bản tin Commit được phát tán nếu nút cục bộ là Token Holder.

if *selfID* $\neq$ *holderID* **then**

 | Kết thúc;

end

Chọn tập Proposal hợp lệ P từ *proposalQueue*;

if P rỗng và không có nhu cầu cập nhật cục bộ **then**

 | Kết thúc;

end

Tạo Commit từ *localState* và P bằng lớp MLS;

if Commit hợp lệ **then**

 | Phát tán Commit cho toàn nhóm;

end

else

 | Loại bỏ các Proposal không còn hợp lệ và ghi nhận lỗi;

end

Trong nguyên lý này, MLS chịu trách nhiệm kiểm tra tính hợp lệ mật mã của Commit, còn lớp điều phối chịu trách nhiệm quyết định ai được quyền phát hành Commit ở Epoch đó. Việc tách ranh giới như vậy giúp giữ rõ trách nhiệm của từng tầng.

0.2.5 Chuyển quyền và failover

Sau mỗi Commit hợp lệ, nhóm chuyển sang Epoch $E + 1$ và mọi nút tự tính lại Token Holder từ trạng thái mới. Vì quy tắc chọn phụ thuộc vào epoch, quyền phát hành Commit có thể luân chuyển giữa các thành viên thay vì gán cố định vào một nút.

Trong mạng P2P, Token Holder có thể ngoại tuyến hoặc không kịp phát hành Commit. Để tránh bế tắc, các nút duy trì một danh sách các thành viên được ghi nhận là đang hoạt động trong nhóm, gọi là ActiveView. Nếu Token Holder hiện tại không còn được quan sát là đang hoạt động hoặc vượt quá thời gian chờ hợp lý, các nút sẽ loại nó khỏi tập ứng viên của Epoch đó và tính lại Token Holder.

Failover này không phải là cơ chế đồng thuận Byzantine đầy đủ. Khi mạng bị

chia cắt, các phân vùng có thể có *ActiveView* khác nhau và tính ra các Token Holder khác nhau. Vai trò của Single-Writer là giảm nguy cơ concurrent commits khi các nút còn chung *ActiveView*, chứ không loại bỏ hoàn toàn khả năng fork trong mọi điều kiện mạng.

0.3 Kiểm tra nhất quán Epoch

0.3.1 Mục tiêu thiết kế

MLS yêu cầu mỗi Commit và thông điệp liên quan phải được xử lý trên đúng trạng thái nhóm tương ứng **rfc9420**. Nếu nút áp dụng bản tin lên trạng thái không tương thích, kết quả có thể là lỗi giải mã, lỗi xác thực, hoặc làm hỏng tiến trình tiến hóa trạng thái cục bộ. Vì vậy, đề án đặt một lớp kiểm tra ngoài MLS để phân loại thông điệp trước khi chuyển xuống tầng mật mã.

Trong mạng P2P bất đồng bộ, thông điệp có thể đến chậm, đến sớm, bị lặp hoặc sai thứ tự. Lớp điều phối cần xác định bản tin thuộc epoch hiện tại, quá khứ hay tương lai trước khi xử lý. Sau khi phân loại, nút mới quyết định chuyển bản tin xuống MLS, từ chối, hay đệm lại để đồng bộ.

0.3.2 Cấu trúc bản tin điều phối

Để phân loại thông điệp, đề án dùng một cấu trúc bản tin điều phối như trong Mã nguồn 1. Đây chỉ là mô hình khái niệm để giải thích vai trò từng trường thông tin; ứng dụng thử nghiệm có thể mã hóa khác đi miễn là giữ nguyên ý nghĩa logic.

Listing 1: Mô hình khái niệm của lớp bao điều phối

```
1 type CoordinationEnvelope struct {
2     GroupID      []byte
3     SenderID     PeerID
4     Type         MessageType
5     Epoch        uint64
6     Timestamp    HLCTimestamp
7     Payload      []byte
8 }
```

Trong đó, *groupID* cho biết bản tin thuộc nhóm nào, *senderID* là nút gửi, *type* cho biết đây là Proposal, Commit hay application message, còn *epoch* cho biết trạng thái MLS mà bản tin được tạo ra. Trường *timestamp* chỉ dùng sắp xếp thông điệp ứng dụng, không dùng để quyết định Commit nào hợp lệ.

0.3.3 Phân loại thông điệp theo Epoch

Khi nhận một bản tin, nút cục bộ không xử lý ngay mà trước hết so sánh *env.epoch* với epoch cục bộ của nhóm tương ứng. Có ba tình huống chính. Nếu

hai giá trị bằng nhau, bản tin thuộc về trạng thái hiện tại và có thể được xử lý theo luồng bình thường. Nếu *env.epoch* nhỏ hơn epoch cục bộ, bản tin thuộc về quá khứ và không được phép kéo trạng thái nhóm lùi lại. Nếu *env.epoch* lớn hơn epoch cục bộ, nút cục bộ được xem là đang đi sau và phải thực hiện đồng bộ hóa thay vì tăng epoch một cách nhảy cóc.

Thuật toán 3 mô tả bước phân loại cơ bản này.

Algorithm 3: Phân loại bản tin theo epoch (*ProcessIncomingEnvelope*)

Đầu vào: *env*: một bản tin ở tầng điều phối; *localEpoch*: epoch cục bộ của nhóm.

Đầu ra : Hành động xử lý tương ứng.

if *env.epoch* < *localEpoch* **then**

 | Từ chối áp dụng bản tin lên trạng thái hiện tại;

else

 | **if** *env.epoch* > *localEpoch* **then**

 | Đưa bản tin vào bộ đệm thông điệp tương lai;

 | Kích hoạt quá trình đồng bộ trạng thái;

 | **else**

 | Chuyển bản tin sang luồng xử lý tương ứng với loại của nó;

 | **end**

end

Cách phân loại này giữ cho epoch cục bộ của mỗi nút là một đại lượng đơn điệu không giảm. Nút không tự ý tăng epoch chỉ vì nhìn thấy bản tin từ tương lai, mà chỉ tăng epoch sau khi đã xử lý thành công các bước chuyển trạng thái cần thiết.

0.3.4 Đồng bộ trạng thái khi gặp thông điệp tương lai

Một thông điệp đến từ tương lai không tự nó là bằng chứng của phân nhánh. Trong nhiều trường hợp, điều đó chỉ cho thấy nút cục bộ đã bỏ lỡ một hoặc nhiều Commit trung gian. Vì vậy, phản ứng đúng của lớp điều phối không phải là kết luận ngay rằng hệ thống đã fork, mà là trước hết kiểm tra xem lịch sử Commit của nút cục bộ có còn là tiền tố hợp lệ của lịch sử trên nhánh xa hay không. Cơ chế kiểm tra này dựa trên một công thức lịch sử Commit, sẽ được trình bày chi tiết ở mục 0.4.

Nếu kiểm tra tiền tố cho thấy lịch sử cục bộ vẫn nằm trên cùng một chuỗi với nhánh còn lại kia, nút chỉ cần áp dụng tuần tự các Commit còn thiếu để bắt kịp. Nếu lịch sử đã phân kỳ, nút mới chuyển sang quy trình Fork Healing. Cả hai trường hợp đều dựa trên cùng một nguyên tắc: một nút không được nhảy cóc sang trạng thái mới nếu chưa có một con đường hợp lệ để đi từ trạng thái hiện tại của nó sang

trạng thái đó.

0.3.5 Xử lý thao tác bị bỏ lỡ

Trong mạng bất đồng bộ, một Proposal do người dùng tạo ra có thể không được đưa vào Commit của Epoch mà nó hướng tới, chẳng hạn vì Token Holder đã chốt một Commit khác trước đó. Đồ án xử lý tình huống này theo nguyên tắc đề xuất lại có điều kiện. Cụ thể, nếu sau khi nhóm chuyển sang Epoch mới mà thao tác của người dùng vẫn còn hợp lệ về mặt ngữ nghĩa, nút cục bộ có thể tạo lại Proposal trên trạng thái mới rồi phát tán lại. Nếu thao tác không còn hợp lệ nữa, ứng dụng phải báo rõ cho người dùng thay vì tiếp tục giữ một ý định đã lỗi thời.

Nguyên tắc này giúp tránh việc đánh đồng giữa một Proposal bị bỏ lỡ và một thao tác phải bị hủy bỏ vĩnh viễn. Đồng thời, nó cũng giữ cho quyền quyết định Commit vẫn nằm ở Token Holder của từng epoch, thay vì cho phép các thao tác cũ tự ý chen vào một trạng thái nhóm đã thay đổi.

0.4 Cơ chế Fork Healing trạng thái

0.4.1 Bối cảnh phân vùng mạng

Single-Writer hoạt động tốt nhất khi các nút còn có cùng ActiveView. Tuy nhiên, trong mạng P2P, phân vùng mạng có thể chia nhóm thành nhiều phân vùng không liên lạc được với nhau. Khi đó, mỗi phân vùng chỉ quan sát được một phần thành viên đang hoạt động và có thể tính ra các Token Holder khác nhau. Nếu các phân vùng tiếp tục tiến hóa độc lập, hệ thống có thể xuất hiện nhiều nhánh trạng thái MLS khác nhau.

Đồ án xem phân vùng mạng là trường hợp phải xử lý ngay từ thiết kế chứ không phải ngoại lệ hiếm. Vấn đề không phải là ngăn hoàn toàn phân nhánh trong môi trường bất đồng bộ – điều đó không khả thi – mà là nhận ra sự phân kỳ và đưa hệ thống hội tụ lại khi các phân vùng nối thông.

0.4.2 Phát hiện nghi vấn phân nhánh và đối chiếu trạng thái

Để nhận biết tình trạng phân kỳ, mỗi nút định kỳ công bố một bản tóm tắt trạng thái nhánh đang giữ. Ở mức khái niệm, bản tóm tắt này có thể được mô tả như Mã nguồn 2.

Listing 2: Bản tóm tắt trạng thái nhánh dùng cho đối chiếu

```
1 type GroupStateAnnouncement struct {
2     GroupID          []byte
3     Epoch            uint64
4     TreeHash         []byte
5     HistoryHash      []byte // R(E) = H(R(E-1) ||
                          ↪ CommitHash(E))
```

```

6     ActiveMemberCount uint32
7 }

```

Ý nghĩa của thông điệp này không phải là thay thế toàn bộ trạng thái MLS, mà là cung cấp một cách đủ ngắn gọn để lớp điều phối có thể so sánh các nhánh với nhau. Trong đó, epoch cho biết mức tiến hóa của nhánh, tree hash phản ánh trạng thái cây MLS, còn history hash là giá trị tóm tắt toàn bộ chuỗi Commit mà nút đã chấp nhận từ epoch 1 đến epoch hiện tại.

Để tính history hash, mỗi nút duy trì một giá trị theo công thức:

$$R(0) = r_0, \quad R(e) = H(R(e-1) \parallel C(e))$$

trong đó r_0 là hằng số khởi tạo, $C(e)$ là giá trị băm của Commit tại epoch e , còn H là hàm băm mật mã. Giá trị $R(E)$ tại epoch hiện tại phản ánh toàn bộ lịch sử Commit từ epoch 1 đến epoch E . Nếu hai nút có cùng $R(e)$ tại cùng một mốc e , thì trong phạm vi giả định rằng các Commit được chuẩn hóa và băm theo cùng một quy tắc, có thể xem như chúng đã đi qua cùng một chuỗi các thao tác lịch sử để cùng đến mốc đó.

Dựa vào cơ chế này, việc phân biệt giữa chậm đồng bộ và phân nhánh chỉ cần đưa về một phép kiểm tra tiền tố. Xét hai nút A và B với $E_A \leq E_B$. Nếu $R_B(E_A) = R_A(E_A)$, tức là giá trị tại epoch mới nhất của chuỗi ngắn hơn trùng khớp với đoạn tương ứng trong chuỗi dài hơn, thì A chỉ đang đi sau trên cùng một nhánh, không có phân nhánh. Ngược lại, nếu $R_B(E_A) \neq R_A(E_A)$, thì hai nút đã đi qua những thao tác lịch sử khác nhau và phân nhánh thực sự đã xảy ra.

Trong trường hợp hai nút đang ở cùng epoch, việc so sánh trực tiếp hai giá trị $R(E)$ là đủ để kết luận. Trong trường hợp một nút có epoch thấp hơn, nút đó cần thêm một lượt hỏi–đáp để lấy giá trị R của nút kia tại epoch của mình rồi mới đối chiếu.

0.4.3 Hàm trọng số nhánh

Khi hệ thống đã xác định rằng đang tồn tại nhiều nhánh khác nhau của cùng một nhóm, nó cần một quy tắc tất định để chọn nhánh tiếp tục. Đồ án dùng một hàm trọng số nhánh với thứ tự ưu tiên từ trái sang phải như sau:

$$W(B) = (C_{active}(B), E(B), H_{commit}(B)) \quad (3)$$

Trong đó, $C_{active}(B)$ là số thành viên đang hoạt động quan sát được trên nhánh

B , $E(B)$ là epoch hiện tại của nhánh, còn $H_{commit}(B)$ là giá trị băm của Commit gần nhất gắn với nhánh đó. Các nhánh được so sánh theo thứ tự từ điển. Nhánh có nhiều thành viên hoạt động hơn được ưu tiên trước; nếu bằng nhau, nhánh có epoch cao hơn được ưu tiên; nếu vẫn bằng nhau, giá trị băm Commit đóng vai trò phá hòa tất định.

Thuật toán 4 mô tả quy tắc so sánh này.

Algorithm 4: So sánh trọng số nhánh (*CompareBranchWeight*)

Đầu vào: B_a , B_b : hai nhánh cần so sánh.

Đầu ra : Nhánh được chọn làm nhánh tiếp tục.

if $C_{active}(B_a) \neq C_{active}(B_b)$ **then**

 Chọn nhánh có C_{active} lớn hơn;

 Kết thúc;

end

if $E(B_a) \neq E(B_b)$ **then**

 Chọn nhánh có epoch lớn hơn;

 Kết thúc;

end

 Chọn nhánh có H_{commit} lớn hơn theo thứ tự từ điển;

Quy tắc này không nói rằng nhánh được chọn là nhánh “đúng” theo nghĩa tuyệt đối. Nó chỉ bảo đảm rằng nếu các nút cùng quan sát một tập nhánh và áp dụng cùng quy tắc tất định, chúng sẽ đi đến cùng một quyết định.

0.4.4 Quy trình Fork Healing

Sau khi chọn được nhánh thắng, hệ thống không cố hợp nhất trực tiếp hai trạng thái MLS đã phân kỳ. Trộn khóa, trộn transcript hay ghép thủ công hai trạng thái như vậy nằm ngoài mô hình vận hành hợp lệ của MLS. Thay vào đó, nút ở nhánh thua lấy thông tin của nhánh thắng rồi gia nhập lại bằng External Proposal **rfc9420**.

Khi phát hiện mình ở nhánh thua, nút đầu tiên hủy trạng thái MLS cũ đang giữ trong bộ nhớ và tạo một KeyPackage mới. Sau đó, nút bọc KeyPackage này vào một External Proposal kiểu Add và phát qua GossipSub. Token Holder của epoch hiện tại nhận được proposal này, kiểm tra xem định danh cũ của nút còn nằm trong cây MLS hay không. Nếu còn, Token Holder tạo thêm một Remove Proposal cho lá cũ và buffer cả Remove lẫn Add vào hàng đợi Single-Writer. Khi khoảng thời gian gom đề xuất hết hạn, Token Holder tạo một Commit duy nhất chứa tất cả proposals đã gom, sinh thông điệp Welcome và chuyển tiếp cho nút nhánh thua. Nút nhánh thua nhận Welcome, gọi ProcessWelcome để khôi phục trạng thái nhóm mới, rồi

tiến hành tự phát lại các tin nhắn do chính nó tạo ra trong thời gian phân vùng.

Quy trình tổng quát được mô tả trong Thuật toán 5.

Algorithm 5: Quy trình Fork Healing (*ForkHealing*)

Đầu vào: *Branches*: tập các nhánh đang được quan sát sau khi đã đối chiếu trạng thái.

Đầu ra : Một trạng thái MLS hợp lệ chung để nhóm tiếp tục vận hành.

winningBranch $\leftarrow$ *SelectBranch(Branches)*;

for mỗi nút đang ở nhánh thua **do**

 Hủy trạng thái MLS cũ;

 Tạo KeyPackage mới;

 Gửi External Proposal (Add) qua GossipSub;

end

Token Holder::

Nhận External Proposal;

if định danh cũ còn trong cây **then**

 Tạo Remove Proposal cho lá cũ;

end

Tạo Add Proposal và buffer vào hàng đợi;

Khi hết hạn gom: tạo Commit chứa tất cả proposals;

Sinh Welcome và chuyển tiếp cho nút nhánh thua;

Nút nhánh thua::

ProcessWelcome $\rightarrow$ khôi phục trạng thái;

Tự phát lại tin nhắn do chính mình tạo (Autonomous Replay);

Nếu nhiều nút cùng bị rơi vào nhánh thua và mỗi nút tự tạo Commit để gia nhập, hệ thống sẽ đối mặt với hiện tượng N Commits cạnh tranh trong cùng epoch, phần lớn bị từ chối và phải thử lại. Việc dùng External Proposal kết hợp với Token Holder gom tất cả đề nghị vào một Commit duy nhất giúp tránh tình huống này. Dù có K nút cùng cần heal, hệ thống chỉ tạo đúng một Commit.

Sau khi quá trình Fork Healing hoàn tất và các nút nhánh thua đã thực hiện gia nhập lại thành công, hệ thống phải đối mặt với một vấn đề mang tính nền tảng của mật mã mã hóa đầu cuối là Backward Secrecy. Do đặc điểm của cơ chế gia nhập lại qua External Proposal, các nút từ một nhánh, khi gia nhập vào nhánh còn lại sẽ không sở hữu khóa bí mật của các epoch đã qua trong thời gian phân mảnh mạng của nhánh đó. Điều này dẫn tới hệ quả là chúng không thể giải mã các thông điệp trong lịch sử đã được nhánh kia trao đổi.

Để lấp đầy khoảng trống dữ liệu này mà không phá vỡ mô hình mã hóa đầu cuối, đồ án sử dụng một cơ chế khôi phục thông điệp là khi nhận diện được mốc hợp nhất nhánh, mỗi thiết bị ở cả hai nhánh đều tiến hành quét lại toàn bộ tin nhắn được lưu cục bộ của mình. Hệ thống yêu cầu mỗi nút tự bọc lại các thông điệp do chính nó tạo ra trong thời gian đứt mạng bằng khóa nhóm của epoch mới nhất, sau đó phát lại vào mạng. Quy tắc này đảm bảo dữ liệu được phục hồi mà không vi phạm ranh giới thẩm quyền của MLS, bởi vì thao tác mã hóa được thực hiện bởi chính tác giả gốc của thông điệp, không cho phép bất kỳ nút nào tự ý đại diện cho thông điệp của người khác.

Tuy nhiên, nếu hàng nghìn thông điệp được phát lại thành từng gói tin đơn lẻ ngay sau khi hợp nhất, mạng sẽ đối mặt với hiện tượng bão mạng gây nghẽn nghiêm trọng. Mặc dù về mặt vật lý, tổng dung lượng dữ liệu của các tin nhắn là không thể thay đổi, nhưng thay vì bắt mạng lưới gánh chịu hàng nghìn lần chi phí đóng gói thông điệp và ép CPU chạy thuật toán mã hóa MLS hàng nghìn lần, hệ thống sẽ gom hàng loạt tin nhắn này lại và gộp chung vào các gói tin lớn. Do mạng P2P có giới hạn kích thước gói tin (thường là 1MB), số lượng tin nhắn sẽ được chia thành những khối dữ liệu để đẩy đi. Cách làm này không làm giảm tổng dung lượng, nhưng sẽ giúp triệt tiêu hoàn toàn lượng metadata định tuyến khổng lồ không cần thiết.

0.5 Thứ tự thông điệp ứng dụng bằng Hybrid Logical Clock

0.5.1 Lý do cần HLC

Epoch trong MLS chỉ quyết định thứ tự thay đổi trạng thái mật mã. Nó không tạo ra một thứ tự toàn phần cho các thông điệp ứng dụng được gửi gần đồng thời trong cùng một Epoch. Trong môi trường P2P, hai thiết bị có thể nhận các thông điệp đó theo thứ tự khác nhau do độ trễ mạng, dù cả hai cuối cùng đều giải mã được đầy đủ.

Nếu giao diện chỉ hiển thị theo thời điểm nhận cục bộ, hai người dùng có thể nhìn thấy lịch sử hội thoại theo hai thứ tự khác nhau. Để giảm tình huống đó, đồ án dùng đồng hồ logic lai HLC. HLC kết hợp thành phần thời gian gần với đồng hồ vật lý với một bộ đếm logic, từ đó tạo ra một khóa sắp thứ tự ổn định hơn cho các thông điệp ứng dụng **kulkarni2014hlc**.

$$HLC = (L, C, N) \quad (4)$$

Trong đó, L là thành phần thời gian logic bám theo thời gian vật lý, C là bộ đếm logic dùng khi nhiều sự kiện có cùng L , còn N là định danh nút dùng để phá hòa

một cách tất định.

0.5.2 Tạo timestamp khi gửi tin

Khi gửi một thông điệp ứng dụng, nút cục bộ lấy thời gian vật lý hiện tại rồi so sánh với giá trị L đang lưu. Nếu thời gian vật lý đã tiến lên, L được cập nhật theo thời gian vật lý đó và bộ đếm C trở về 0. Nếu thời gian vật lý chưa tăng, hệ thống chỉ tăng bộ đếm logic. Thuật toán này được mô tả trong Thuật toán 6.

Algorithm 6: Sinh thời gian logic lai (HLC_Now)

Đầu vào: hlc : trạng thái HLC cục bộ; $nodeID$: định danh nút cục bộ.

Đầu ra : Timestamp HLC mới ($L, C, nodeID$).

$pt \leftarrow$ thời gian vật lý hiện tại;

if $pt > hlc.L$ **then**

$hlc.L \leftarrow pt$;

$hlc.C \leftarrow 0$;

end

else

$hlc.C \leftarrow hlc.C + 1$;

end

return ($hlc.L, hlc.C, nodeID$);

0.5.3 Cập nhật HLC khi nhận tin

Khi nhận một thông điệp ứng dụng mang timestamp HLC, nút cục bộ cập nhật đồng hồ của mình theo giá trị lớn nhất giữa đồng hồ cục bộ, timestamp nhận được và thời gian vật lý hiện tại. Cách cập nhật này bảo toàn thứ tự nhân quả cơ bản mà không buộc các thiết bị phải đồng bộ đồng hồ tuyệt đối. Thuật toán tương ứng được mô tả trong Thuật toán 7.

Algorithm 7: Cập nhật đồng hồ logic lai (HLC_Update)

Đầu vào: hlc : trạng thái HLC cục bộ; msg : timestamp HLC nhận được
($msg.L, msg.C, msg.NodeID$).

Đầu ra : Trạng thái HLC cục bộ sau khi cập nhật.

```
 $pt \leftarrow$  thời gian vật lý hiện tại;  
 $newL \leftarrow \max(hlc.L, msg.L, pt);$   
if  $newL == hlc.L$  and  $newL == msg.L$  then  
     $hlc.C \leftarrow \max(hlc.C, msg.C) + 1;$   
end  
else if  $newL == hlc.L$  then  
     $hlc.C \leftarrow hlc.C + 1;$   
end  
else if  $newL == msg.L$  then  
     $hlc.C \leftarrow msg.C + 1;$   
end  
else  
     $hlc.C \leftarrow 0;$   
end  
 $hlc.L \leftarrow newL;$   
return  $hlc;$ 
```

0.5.4 Vai trò của HLC trong hệ thống

HLC chỉ được dùng để sắp thứ tự hiển thị các thông điệp ứng dụng đã vượt qua bước kiểm tra epoch và đã được MLS xử lý hợp lệ. Nó không quyết định Commit nào đúng, không thay thế lớp điều phối ở tầng trạng thái và không được phép ảnh hưởng tới key schedule của MLS. Sự tách vai trò này là điều kiện cần để hệ thống vừa giữ đúng mô hình bảo mật của MLS, vừa có một thứ tự hiển thị ổn định hơn ở tầng ứng dụng.

0.6 Hiện thực ứng dụng thử nghiệm phục vụ kiểm chứng

0.6.1 Ranh giới trách nhiệm trong ứng dụng thử nghiệm

Ở mức hiện thực ứng dụng thử nghiệm, các nguyên tắc trên được ánh xạ thành một kiến trúc hai tầng gồm một tiến trình chủ chịu trách nhiệm điều phối và một thành phần MLS chịu trách nhiệm xử lý mật mã. Trạng thái bền vững của nhóm được lưu ở phía host; khi cần xử lý Proposal, Commit, External Proposal, External Commit hoặc application message, host lấy trạng thái hiện tại, chuyển nó cho tầng MLS xử lý rồi ghi lại trạng thái mới sau khi thao tác hoàn tất.

Cách hiện thực này giữ đúng ranh giới đã nêu: quyết định điều phối nằm ở tầng ngoài, MLS chỉ xử lý các phép biến đổi trạng thái hợp lệ. Việc ứng dụng dùng công nghệ gì ở tầng giao diện không làm thay đổi bản chất của đóng góp nghiên cứu.

0.6.2 Vai trò của ứng dụng desktop

Trong ứng dụng thử nghiệm của đồ án, ứng dụng desktop không được xem là phần đóng góp thuật toán mới. Vai trò của nó là hiện thực hóa các luồng thao tác quan trọng nhất để kiểm chứng phương pháp: tham gia hệ thống bằng bundle, tạo và quản trị nhóm, và trao đổi thông điệp trên nhiều nút.

Vì lý do đó, các màn hình cụ thể, ảnh chụp ứng dụng và kịch bản demo không được trình bày chi tiết ngay trong Chương 3. Thay vào đó, Chương 5 sẽ dùng ứng dụng thử nghiệm như một môi trường kiểm chứng đầu cuối, còn phụ lục sẽ tập hợp đầy đủ các hình minh họa cần thiết cho việc đối chiếu.

0.7 Tính đúng đắn trực giác của phương pháp

Khi các nút còn chung ActiveView, Single-Writer giảm khả năng concurrent commits vì mỗi epoch chỉ có một Token Holder được quyền Commit. Kết hợp với kiểm tra epoch, cơ chế này buộc mọi bước tiến hóa trạng thái cục bộ phải đi qua chuỗi Commit hợp lệ, không cho phép nút tự ý xử lý bản tin trên trạng thái không tương thích.

Khi phân vùng mạng xảy ra, phương pháp không giả định hệ thống giữ được một trạng thái duy nhất mọi lúc. Nó chấp nhận phân kỳ tạm thời, nhưng yêu cầu lớp điều phối nhận ra khác biệt giữa các nhánh, thử bắt kịp khi cần, rồi chọn một nhánh để hội tụ bằng cơ chế hợp lệ của MLS. Hệ thống không cố hợp nhất trực tiếp hai trạng thái đã rẽ nhánh.

HLC giữ thứ tự hiển thị ổn định cho thông điệp ứng dụng sau khi đã xử lý hợp lệ. Phương pháp đề xuất quan tâm cả đến nhất quán trạng thái mật mã lẫn tính quan sát được ở tầng ứng dụng, nhưng vẫn tách rõ hai loại ordering này.

Kết chương. Chương 3 đã trình bày lớp điều phối phi tập trung mà đồ án đề xuất cho MLS trên mạng ngang hàng. Phần cốt lõi của phương pháp gồm bốn cơ chế: Single-Writer theo epoch để hạn chế concurrent commits, kiểm tra nhất quán epoch để bảo vệ trạng thái cục bộ, phát hiện và Fork Healing để hỗ trợ hội tụ sau phân vùng mạng, và HLC để sắp thứ tự thông điệp ứng dụng.

Các cơ chế trên tạo thành phần nghiên cứu trung tâm của đồ án. Ứng dụng thử nghiệm chỉ đóng vai trò hiện thực hóa và kiểm chứng các nguyên tắc đó trong một hệ thống chạy được. Trên cơ sở này, Chương 4 sẽ tiếp tục phân tích các bất biến và giới hạn lý thuyết của phương pháp trước khi Chương 5 chuyển sang phần kiểm

chứng bằng ứng dụng thử nghiệm và thực nghiệm.